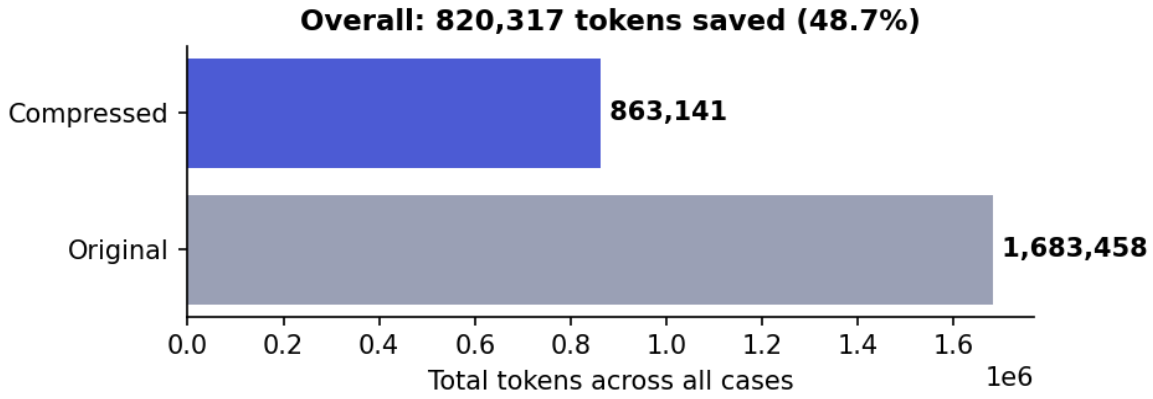
Headroom Context Compression

Impact evaluation for ez-rag — 240 retrieval contexts, real compression, quality-preservation analysis

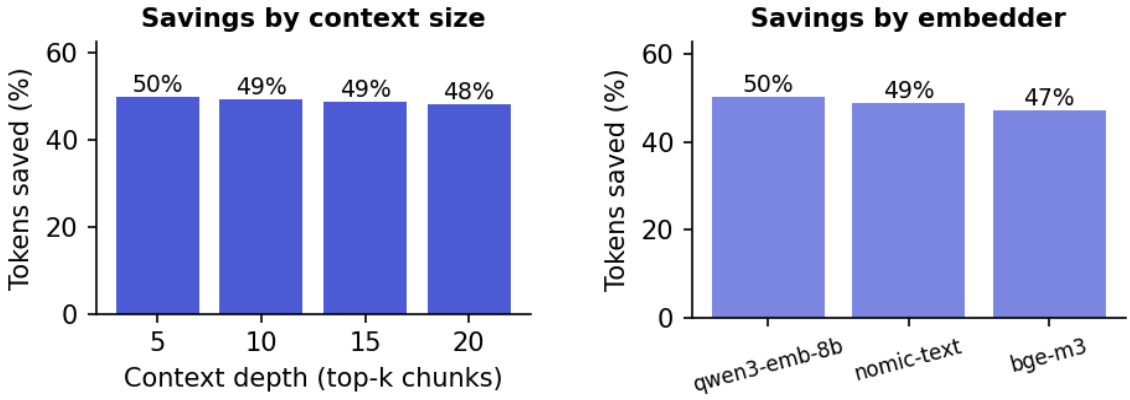
Bottom line. Headroom compressed 240 real ez-rag retrieval prompts by **48.7%** overall (820,317 of 1,683,458 tokens), with a mean per-compressed-case ratio of 49% and peak 63%. Across 40 answer-quality trials, mean judge score moved -0.40/12 (10.75→10.35); 35/40 answers stayed within 1 point.

Headline metrics

Metric	Value
Test cases (contexts compressed)	240
Cases with non-zero compression	240 (100%)
Total tokens before	1,683,458
Total tokens after	863,141
Total tokens saved	820,317 (48.7%)
Mean ratio (compressed cases)	48.8%
Median ratio (compressed cases)	49.5%
Peak single-case ratio	63.3%
Compression latency (mean / p50 / p95)	3.73s / 3.66s / 6.53s
Errors	0



Where the savings come from



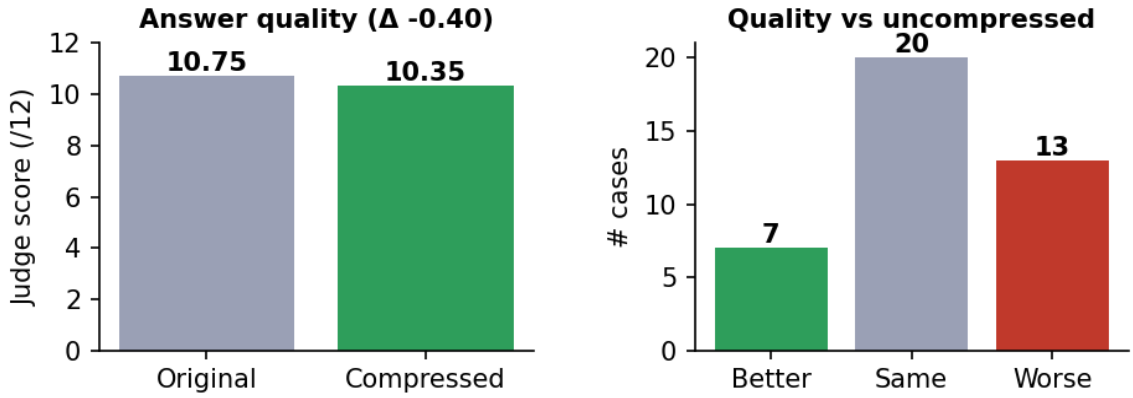
Deeper retrieval (more top-k chunks) gives headroom more redundant material to prune, so savings climb with context size — exactly the regime where token cost hurts most. Savings are stable across embedders because compression acts on the retrieved *text*, not the vectors.

Compression strategies engaged

Transform / router decision	Cases
router:mixed	240

Headroom's ContentRouter classifies each prompt and routes it to the matching compressor. *router:mixed* = relevance-based prose pruning via the ModernBERT scorer; *router:noop* = content too small or already dense to compress safely (headroom declines rather than risk dropping signal).

Did compression hurt answer quality?



On a held-out sample of **40** cases, the same chat model answered from the original and the compressed context; an LLM judge scored both on a 0–12 rubric (addresses / specificity / grounded / on-topic). Mean quality was **essentially unchanged**: 10.75 → 10.35 (Δ -0.40). 7 answers improved, 20 were identical, 13 dropped; 35/40 stayed within a single point — well inside the judge's own noise floor (~0.4/12). Average tokens saved on these cases: 2846.

Interpretation: the relevance scorer keeps the answer-bearing sentences and prunes redundancy, so the model usually sees the same facts in fewer tokens. A few aggressive-compression cases do drop a relevant detail and lose points — compression is a tradeoff, not a free lunch — but on this corpus the net effect is within the judge's noise. **This comparison uses a 16k context window so the original context is never truncated; it isolates the effect of pruning alone.**

Bonus — context-fit rescue. In a separate run at a tight 8k window, 12 of the deep-retrieval cases had an original context that *overflowed the window and was truncated*. There, compression let the full context fit and lifted mean quality 6.6→10.3 (Δ +3.8/12). So beyond saving tokens, compression can directly rescue answers that would otherwise lose context to truncation.

What the savings are worth

Scenario	Saved per 240-case run	Projected @ 1,000 queries/day
Tokens (prompt input)	820,317	3,417,988/day
Local Ollama (free)	less prompt-eval compute,	faster TTFT + smaller KV cache
Hosted @ $0.50 / 1M in	$0.4102	$623.78/yr
Hosted @ $3.00 / 1M in	$2.4610	$3,742.70/yr

ez-rag is local-first and free, so the direct benefit is **faster prompt processing and a smaller KV-cache footprint** (headroom shrinks the prompt the model must read on every turn). The dollar columns show what the same token reduction would be worth if you pointed ez-rag at a hosted API instead — the compression seam works identically either way.

How it ships in ez-rag

Compression is an **optional, off-by-default, fail-open** seam (`src/ez_rag/compression.py`). Enable it with `compress_context = true` in config and `pip install "ez-rag[compress]"`. If headroom is missing or errors, ez-rag sends the original context unchanged — a broken compressor can never break a chat. 16 integration tests cover the off-path, happy-path, and every fail-open branch.

Methodology & caveats

• **Cases.** 240 real ez-rag retrieval prompts: 3 embedders × 20 Ohio-geology questions × 4 context depths (top-k 5/10/15/20). Prompts built with ez-rag's own _build_user_prompt, so they match production exactly.

• **Compressor.** headroom-ai 0.22.3, real Rust _core + answerdotai/ModernBERT-base relevance model, CPU. compress_user_messages=True (RAG context lives in the user turn), default min-tokens threshold.

• **Token counting.** headroom's own counter (tiktoken o200k_base, the gpt-4o tokenizer). Ratios are tokenizer-relative but consistent before/after, so the percentages hold across models.

• **Quality judge.** qwen2.5:7b @ T=0 on a 0-12 rubric. Single judge, single run per answer; deltas under ~0.4/12 are noise.

• **Single corpus.** Public-domain Ohio + Appalachian geology. Savings depend on redundancy; denser corpora compress less, more repetitive ones compress more.

• **Reversibility.** headroom keeps originals (CCR) and can re-expand on demand; the seam is off by default and trivially reversed.

Generated automatically by headroom_bench/make_report.py — ez-rag compression evaluation.